Лабораторная работа №2. Статический анализ кода (SAST) с Semgrep

Тема: поиск опасных конструкций в исходном коде, исправление SQL-инъекций и XSS, подтверждение исправлений тестами. **Время:** 2 академических часа. **Результат:** отчёты Semgrep «до» и «после», исправленный код, тесты, которые подтверждают защиту, и коммит в вашем репозитории.

Перед началом: выполнена ЛР1, есть каталог `~/devsecops-lab` с репозиторием и окружением `.venv`.

---

1. Цель работы

Научиться запускать статический анализатор, читать его находки, отличать настоящие дефекты от ложных срабатываний и устранять причину дефекта, а не предупреждение.

После работы вы сможете:

- объяснить, что такое источник (source) и приёмник (sink) недоверенных данных;
- прочитать правило Semgrep и понять, что именно оно ищет;
- заменить склейку SQL-строк параметризованным запросом;
- объяснить, почему пустой отчёт сканера не означает, что код безопасен.

2. Теория

2.1. Что такое SAST

SAST (Static Application Security Testing) — анализ исходного кода **без его запуска**. Анализатор читает файлы, строит внутреннее представление программы и ищет в нём конструкции, которые считаются опасными.

Сильные стороны	Ограничения
не нужно запускать приложение	не видит ошибок логики: кто имеет право на что
показывает точный файл и строку	даёт ложные срабатывания
работает за секунды, подходит для каждого коммита	находит только то, на что написаны правила
видит весь код, включая редко исполняемые ветки	не знает, как приложение настроено при запуске

2.2. Источник, приёмник и обезвреживание

Большинство уязвимостей веб-приложений устроены одинаково: недоверенные данные доходят до опасной операции без обработки.

- **Источник (source)** — место, где в программу попадают данные от пользователя. В нашем приложении это `request.form` (поля формы) и `request.args` (параметры адресной строки).
- **Приёмник (sink)** — операция, для которой содержимое данных критично: выполнение SQL (`cursor.execute`), вставка в HTML-страницу, запуск команды операционной системы.
- **Обезвреживание (sanitizer)** — механизм, после которого данные для приёмника безопасны: параметры запроса для SQL, экранирование для HTML.

Задача аналитика — для каждой находки проследить путь «источник → приёмник» и понять, есть ли на нём обезвреживание.

2.3. SQL-инъекция

Посмотрите на строку из функции `login`:

```
query = f"SELECT id, username, role FROM users WHERE username = '{username}' AND
 password = '{password}'"
```

Текст запроса и данные пользователя склеиваются в одну строку. Если ввести логин `admin' --` , база данных получит:

```
SELECT id, username, role FROM users WHERE username = 'admin' -- ' AND password
= '...'
```

Кавычка закрыла строку раньше времени, а `--` в SQL начинает комментарий. Проверка пароля исчезла из запроса: данные пользователя стали частью команды.

Защита — **параметризованный запрос**. Текст команды и значения передаются базе отдельно:

```
cursor.execute("SELECT ... WHERE username = ? AND password = ?", (username, pass
word))
```

Теперь база знает, где команда, а где данные. Что бы пользователь ни ввёл, это останется строкой для сравнения.

2.4. XSS — межсайтовое выполнение сценариев

XSS (Cross-Site Scripting) возникает, когда текст пользователя вставляется в HTML-страницу как разметка. Если в комментарий записать тег `<script>`, браузер каждого посетителя выполнит этот сценарий — от имени сайта и с доступом к данным посетителя.

- **Отражённый XSS** — вредоносный текст приходит в запросе и сразу возвращается в ответе (страница поиска).

- **Хранимый XSS** — текст сохраняется в базе и показывается всем (комментарии). Он опаснее: жертве не нужно переходить по специальной ссылке.

Защита — **экранирование**: символы <, >, &, кавычки заменяются на безопасные последовательности (`<`, `>` и так далее), и браузер показывает их как текст. Шаблонизатор Jinja2, который использует Flask, экранирует значения автоматически. Фильтр `|safe` это экранирование **отключает** — именно он и создаёт уязвимость в нашем приложении.

2.5. Как устроено правило Semgrep

Откройте файл `.semgrep.yml`. Каждое правило — это шаблон кода. Например:

```yaml
- id: lab.sql-injection-fstring-direct
  languages: [python]
  severity: ERROR
  message: SQL создаётся через f-string непосредственно в execute.
  pattern: $CURSOR.execute(f"...")
```

- `$CURSOR` — метапеременная: на этом месте может стоять любое выражение;
- `f"..."` — любая f-строка;
- правило сработает на любой вызов `что-угодно.execute(f"...")` с одним аргументом.

Semgrep сравнивает не текст, а структуру кода, поэтому пробелы и переносы строк на результат не влияют.

2.6. Ложные срабатывания и пропуски

- **Ложное срабатывание (false positive, FP)** — сканер сообщил о проблеме, которой нет.
- **Пропуск (false negative, FN)** — проблема есть, но сканер её не заметил.

Оба случая неизбежны. Поэтому каждую находку **разбирают** и принимают одно из решений: исправить, признать ложной с письменным обоснованием или принять риск. Пустой отчёт означает лишь «по этим правилам ничего не найдено».

ГОСТ Р 71207-2024 требует от статического анализа воспроизводимости: фиксируются версия анализатора, набор правил, объект проверки и решение по каждому срабатыванию. В отчёте вы делаете то же самое в учебном масштабе.

3. Ход работы

Все команды выполняются в каталоге `~/devsecops-lab` с активированным окружением:

```
cd ~/devsecops-lab
source .venv/Scripts/activate      # Windows, Git Bash
# source .venv/bin/activate        # Linux и macOS
```

Шаг 1. Запустить Semgrep и получить исходный отчёт

```
semgrep --version
mkdir -p reports
semgrep scan --config .semgrep.yml --metrics=off --json-output reports/sast-befo
re.json .
```

Что означают параметры:

- `--config .semgrep.yml` — использовать правила из этого файла;
- `--metrics=off` — не отправлять статистику разработчикам Semgrep;
- `--json-output` — дополнительно сохранить результат в файл для отчёта;
- `.` — проверять текущий каталог.

Ожидаемый результат: в конце вывода `Ran 5 rules on … files: 6 findings.`

Правило	Файл	Строка
`lab.github-token-literal`	`backend/config.py`	3
`lab.sql-injection-fstring-variable`	`backend/app.py`	37
`lab.sql-injection-fstring-direct`	`backend/app.py`	54
`lab.flask-debug-enabled`	`backend/app.py`	94
`lab.template-safe-filter`	`frontend/templates/comments.html`	14
`lab.template-safe-filter`	`frontend/templates/search.html`	9

Если не получилось:

- **`semgrep: command not found`** — не активировано окружение `.venv`.
- **Ошибка про `semgrep-core`** — установлена старая версия. Выполните `python -m pip install -r requirements-dev.txt` и проверьте, что `semgrep --version` показывает 1.185.0.
- **Запасной вариант через Docker** (в Git Bash сначала выполните `export MSYS_NO_PATHCONV=1`):

  ```
  docker run --rm -v "$PWD:/src" semgrep/semgrep:1.185.0 semgrep scan --conf
  ig /src/.semgrep.yml --metrics=off --json-output /src/reports/sast-before.
  json /src
  ```

В отчёт: версия Semgrep, команда запуска, итоговая строка вывода.

Шаг 2. Разобрать находки

Для каждой из шести находок откройте указанный файл на указанной строке и заполните таблицу. Первая строка — образец.

№	Правило	Файл и строка	Источник данных	Приёмник	Настоящий дефект или FP	Чем грозит
1	`sql-injection-fstring-variable`	`app.py:37`	`request.form["username"], ["password"]`	`cursor.execute(query)`	настоящий	вход без пароля, чтение таблицы `users`
2						
…						

Для находок, у которых нет пути «источник → приёмник» (отладочный режим, токен), в этих колонках поставьте прочерк и опишите угрозу своими словами.

Теперь ответьте на вопрос, который показывает ограничения инструмента. В `app.py` есть ещё два вызова `execute` с f-строкой — в функциях `comments` и `profile`:

```
connection.cursor().execute(
    f"INSERT INTO comments (author, content) VALUES ({mark}, {mark})",
    (author, content),
)
```

Почему Semgrep о них не сообщил и правильно ли это? Подсказка: посмотрите, что подставляется в f-строку и сколько аргументов у `execute`.

В отчёт: таблица находок и ответ на вопрос.

Шаг 3. Убедиться, что SQL-инъекция настоящая

Запустите приложение в отдельном терминале (`python -m backend.app`) и откройте http://127.0.0.1:5000/login.

1. Введите логин `admin' --` (после двух дефисов обязательно пробел) и любой пароль.
2. Вы вошли как `admin`, не зная пароля.
3. Выйдите из системы. В поиске введите `' OR '1'='1` — в результатах появятся все материалы, хотя такого текста нет ни в одном.

В отчёт: снимок экрана после входа с логином `admin' --` и объяснение в одном-двух предложениях, почему проверка пароля не сработала.

Шаг 4. Исправить SQL-инъекцию в `/login`

Откройте `backend/app.py`, найдите функцию `login` и эти две строки:

```
query = f"SELECT id, username, role FROM users WHERE username = '{username}' AND
 password = '{password}'"
user = database.get_db().cursor().execute(query).fetchone()
```

Замените их на:

```
mark = database.placeholder()
user = database.get_db().cursor().execute(
    f"SELECT id, username, role FROM users WHERE username = {mark} AND password
= {mark}",
    (username, password),
).fetchone()
```

Что изменилось:

- `database.placeholder()` возвращает знак параметра: `?` для SQLite и `%s` для PostgreSQL. В f-строку теперь подставляется только он;
- значения `username` и `password` передаются вторым аргументом — кортежем. Драйвер базы данных сам подставит их как данные.

Сохраняйте отступы: в Python они часть синтаксиса. Новые строки должны начинаться на том же уровне, что и заменённые.

Ожидаемый результат: вход `student / student123` работает, вход с логином `admin' --` даёт сообщение «Неверные учётные данные». Сервер в отладочном режиме перезапускается сам после сохранения файла.

Шаг 5. Исправить SQL-инъекцию в `/search`

В функции `search` найдите строку:

```
cursor.execute(f"SELECT id, title, content FROM posts WHERE title LIKE '%{query_
text}%' OR content LIKE '%{query_text}%'")
```

Замените её на:

```
pattern = f"%{query_text}%"
mark = database.placeholder()
cursor.execute(
    f"SELECT id, title, content FROM posts WHERE title LIKE {mark} OR content LI
KE {mark}",
    (pattern, pattern),
)
```

Знаки `%` для поиска по подстроке теперь добавляются к значению в Python, а не вписываются в текст запроса.

Ожидаемый результат: поиск по слову Безопасная находит один материал; запрос `' OR '1'='1` не находит ничего.

Шаг 6. Исправить XSS и отключить отладочный режим

1. В файле `frontend/templates/search.html` замените `{{ query|safe }}` на `{{ query }}`.
2. В файле `frontend/templates/comments.html` замените `{{ comment[2]|safe }}` на `{{ comment[2] }}`.
3. В последней строке `backend/app.py` замените `debug=True` на `debug=False`.

После третьей правки остановите сервер (`Ctrl+C`) и запустите заново: без отладочного режима он больше не перезапускается сам.

Проверьте в браузере: введите в поиске `<b>проверка</b>`. Раньше слово выделялось жирным, теперь теги видны как текст.

Почему отладочный режим опасен: при ошибке Flask показывает в браузере страницу с исходным кодом и интерактивной консолью Python. Тот, кто получит к ней доступ, сможет выполнять команды на вашем компьютере.

В отчёт: фрагменты кода «было → стало» для шагов 4–6. Удобно взять их из вывода команды `git diff`.

Шаг 7. Переписать тесты

Сейчас тесты проверяют, что уязвимости **есть**. После исправлений часть из них должна упасть — убедитесь:

```
pytest -q
```

Ожидаемый результат: `2 failed, 2 passed`. Это правильно: поведение изменилось, а тесты описывают старое.

Откройте `tests/test_app.py`.

Замените функцию `test_login_sql_injection_is_demonstrable` на три теста:

```
def test_valid_login_still_works(client):
    response = client.post("/login", data={
        "username": "student",
        "password": "student123",
    })
    assert response.status_code == 302


def test_login_sql_injection_is_blocked(client):
    response = client.post("/login", data={
        "username": "admin' -- ",
        "password": "wrong",
    })
    assert response.status_code == 200
    assert "Неверные учётные данные" in response.get_data(as_text=True)
```

```python
def test_search_treats_quote_as_text(client):
    response = client.get("/search?q=' OR '1'='1")
    assert response.status_code == 200
    assert "Правило лаборатории" not in response.get_data(as_text=True)
```

Замените функцию `test_stored_content_is_rendered_without_escaping` на два теста:

```python
def test_stored_content_is_escaped(client):
    client.post("/comments", data={
        "author": "student",
        "content": "<b>training-xss</b>",
    })
    page = client.get("/comments").get_data(as_text=True)
    assert "<b>training-xss</b>" not in page
    assert "&lt;b&gt;training-xss&lt;/b&gt;" in page


def test_search_query_is_escaped(client):
    page = client.get("/search?q=<b>x</b>").get_data(as_text=True)
    assert "<b>x</b>" not in page
```

Тесты `test_health` и `test_profile_is_available_without_authentication` не трогайте: доступ к профилю вы исправите в ЛР5.

Обратите внимание на первый тест: он проверяет, что обычный вход по-прежнему работает. Исправление безопасности, которое ломает основную функцию, — плохое исправление.

```
pytest -q
```

Ожидаемый результат: `7 passed`.

В отчёт: вывод `pytest -q` до и после изменения тестов.

Шаг 8. Повторить анализ и сравнить

```
semgrep scan --config .semgrep.yml --metrics=off --json-output reports/sast-afte
r.json .
```

Ожидаемый результат: `1 finding` — осталось только `lab.github-token-literal` в `config.py`. Это задача ЛР3.

Запустите ту же проверку так, как её запускают в конвейере CI, — с параметром `--error`, и сразу выведите код завершения:

```
semgrep scan --config .semgrep.yml --metrics=off --error .
echo "Код завершения: $?"
```

Ожидаемый результат: `Код завершения: 1`. Без `--error` Semgrep возвращает 0, даже когда находки есть. С `--error` любая находка даёт ненулевой код — так конвейер понимает, что изменение нужно остановить. Этот приём вы используете в ЛР5.

В отчёт: таблица «было 6 находок → осталась 1» с указанием, что именно осталось и почему.

Шаг 9. Зафиксировать результат

```
git status
git add -A
git commit -m "lab2: fix SQL injection, XSS and debug mode"
git log --oneline
```

Ожидаемый результат: в истории два коммита. Папка `reports` в коммит не попадает — она в `.gitignore`.

4. Требования к отчёту

1. Версия Semgrep, команда и итог первого запуска.
2. Таблица шести находок с источником, приёмником и оценкой.
3. Ответ на вопрос о двух вызовах `execute`, на которые Semgrep не среагировал.
4. Доказательство SQL-инъекции (снимок экрана) и объяснение.
5. Изменения кода «было → стало» (`git diff` или фрагменты).
6. Вывод `pytest` до и после правки тестов.
7. Сравнение отчётов «до» и «после», объяснение кода завершения с `--error`.
8. Вывод: какие уязвимости из вашей модели угроз (ЛР1) Semgrep **не нашёл** и почему.

5. Контрольные вопросы

1. Чем статический анализ отличается от запуска тестов?
2. Что такое источник и приёмник в функции `login`?
3. Почему параметризованный запрос защищает от инъекции, а «удалить кавычки из ввода» — ненадёжный способ?
4. Что делает фильтр `|safe` и когда его применение оправдано?
5. Чем отражённый XSS отличается от хранимого? Какой опаснее и почему?
6. Сканер показал 0 находок. Что из этого следует, а что — нет?
7. Зачем после исправления нужен тест, если сканер уже ничего не находит?

6. Критерии оценивания

Критерий	Баллы
Запуск Semgrep и исходный отчёт	10
Разбор находок: источник, приёмник, оценка	20
Исправление SQL-инъекций	25
Исправление XSS и отладочного режима	15
Тесты, подтверждающие защиту	20
Повторный анализ, выводы, коммит	10
Итого	**100**

7. Источники

- ГОСТ Р 71207-2024. Статический анализ программного обеспечения. Общие требования.
- Документация Semgrep: https://semgrep.dev/docs/
- OWASP SQL Injection Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html
- OWASP Cross Site Scripting Prevention Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html